Singapore Management University

Institutional Knowledge at Singapore Management University

Research Collection School Of Computing and Information Systems | School of Computing and Information Systems

8-2018

Lightweight break-glass access control system for healthcare Internet-of-Things

Yang YANG
Fuzhou University

Ximeng LIU
Singapore Management University, xmliu@smu.edu.sg

Robert H. DENG
Singapore Management University, robertdeng@smu.edu.sg

Follow this and additional works at: https://ink.library.smu.edu.sg/sis_research

 Part of the Information Security Commons, and the Medicine and Health Sciences Commons

Citation

YANG, Yang; LIU, Ximeng; and DENG, Robert H.. Lightweight break-glass access control system for healthcare Internet-of-Things. (2018). *IEEE Transactions on Industrial Informatics*. 14, (8), 3610-3617.
Available at: https://ink.library.smu.edu.sg/sis_research/4151

This Journal Article is brought to you for free and open access by the School of Computing and Information Systems at Institutional Knowledge at Singapore Management University. It has been accepted for inclusion in Research Collection School Of Computing and Information Systems by an authorized administrator of Institutional Knowledge at Singapore Management University. For more information, please email cherylds@smu.edu.sg.

Published in IEEE Transactions on Industrial Informatics, August 2018, Volume 14, Issue 8, pp. 3610-3617.
https://doi.org/10.1109/TII.2017.2751640

Lightweight Break-Glass Access Control System for Healthcare Internet-of-Things

Yang Yang, *Member, IEEE*, Ximeng Liu, *Member, IEEE*, and Robert H. Deng, *Fellow, IEEE*

***Abstract*—Healthcare Internet-of-things (IoT) has been proposed as a promising means to greatly improve the efficiency and quality of patient care. Medical devices in healthcare IoT measure patients' vital signs and aggregate these data into medical files which are uploaded to the cloud for storage and accessed by healthcare workers. To protect patients' privacy, encryption is normally used to enforce access control of medical files by authorized parties while preventing unauthorized access. In healthcare, it is crucial to enable timely access of patient files in emergency situations. In this paper, we propose a lightweight break-glass access control (LiBAC) system that supports two ways for accessing encrypted medical files: attribute-based access and break-glass access. In normal situations, a medical worker with an attribute set satisfying the access policy of a medical file can decrypt and access the data. In emergent situations, the break-glass access mechanism bypasses the access policy of the medical file to allow timely access to the data by emergency medical care or rescue workers. LiBAC is lightweight since very few calculations are executed by devices in the healthcare IoT network, and the storage and transmission overheads are low. LiBAC is formally proved secure in the standard model and extensive experiments are conducted to demonstrate its efficiency.**

***Index Terms*—Access control, break-glass, healthcare system, lightweight, secure Internet-of-Things (IoT).**

Manuscript received July 19, 2017; revised August 18, 2017; accepted August 24, 2017. Date of publication September 13, 2017; date of current version August 1, 2018. This work is supported in part by the National Natural Science Foundation of China under Grant 61402112 and Grant 61702105, in part by Singapore National Research Foundation under Grant NRF2014NCR-NCR001-012, in part by AXA Research Fund, in part by Fujian Provincial Key Laboratory of Information Processing and Intelligent Control, Minjiang University, under Grant MJUKF201734, in part by Fujian Major Project of Regional Industry under Grant 2014 H4015, and in part by Major Science and Technology Project of Fujian Province under Grant 2015H6013. Paper no. TII-17-1597. *(Corresponding author: Ximeng Liu.)*

Y. Yang is with the College of Mathematics and Computer Science, Fuzhou University, Fuzhou 350116, China, with the School of Information Systems, Singapore Management University, Singapore 188065, with the University Key Laboratory of Information Security of Network Systems, Fuzhou University, Fujian 350116, China, and also with the Fujian Provincial Key Laboratory of Information Processing and Intelligent Control, Minjiang University, Fuzhou 350121, China (e-mail: yang.yang.research@gmail.com).

X. Liu is with the College of Mathematics and Computer Science, Fuzhou University, Fuzhou 350116, China, and also with the School of Information Systems, Singapore Management University, Singapore 188065 (e-mail: snbnix@gmail.com).

R. H. Deng is with the School of Information Systems, Singapore Management University, Singapore 188065 (e-mail: robertdeng@smu.edu.sg).

I. Introduction

INTERNET-OF-THINGS (IoT) is a novel paradigm [1], [2] for modern pervasive wireless communications, which connects a variety of physical devices through the Internet to collect and exchange data. A healthcare IoT network [3], [4] consisting of smart sensors on wearable health devices and medical instruments (such as heart monitoring implants and biochip transponders) can remotely monitor a patient's health conditions. The wearable or implantable sensors in healthcare IoT are normally equipped with very limited battery supply. Frequent charging of many sensors and mobile medical devices may fatigue the frail patients and the engaged nurses, which greatly reduces user experience. Furthermore, some implantable body sensors (such as cardiac pacemaker) are difficult or even impossible to be charged, and the patient's life is threatened when the wireless medical devices ran out of power. Thus, operations in the healthcare IoT should be lightweight to save the battery.

Medical devices in healthcare IoT measure patients' vital signs and aggregate these data into a medical file. The storage capability of health IoT is limited and the accumulative medical files have to be stored in a third party storage. Cloud computing [5], [6] is a new Internet-based computing paradigm to provide shared processing and storage resources to customers, and enables ubiquitous and on-demand data access. The medical files are suitable to be stored in the cloud platform to save the local storage cost and enjoy the convenient data access service. Since medical files contain sensitive physiological data, access control over outsourced files is essential to prohibit unauthorized data access.

Standard access control mechanisms, however, are not designed to deal with emergency situations, which are quite common in healthcare. In emergency situations, for example, when a patient suffers from a heart attack or suddenly fainted, the patient loses consciousness and is unavailable to grant access privilege to the first-aid rescuers, which may lead to delayed treatment or even death. It is crucial to realize a break-glass access mechanism for the emergency situations in order to save patient's lives [7]. The term "break-glass" means that someone is not designated to execute the task but gains authorization for the task in exceptional scenarios, which is derived from the action of breaking the glass and ringing the fire alarm [23]. Break-glass access is a mechanism to bypass the normal access policy and gain emergency access to patient records. Since the break-glass authority overrides the access policy and can freely access patient's medical data, the system users with constrained access privilege may struggle to get the break-glass right in

order to break through the constraint of the access policy. Thus, the break-glass authority should be controlled and not abused.

Envisioning these specific requirements in the healthcare IoT network, we propose a lightweight break-glass access control (LiBAC) system, which simultaneously supports two types of access control patterns: attribute-based access control for normal circumstance and break-glass access for emergency circumstance. The main contributions of the paper are listed as follows.

1) *Patient controlled encryption:* In LiBAC, the healthcare records are encrypted by the patients themselves. The patient controlled encryption enables patients to enforce access control policy in order to securely share records among medical workers, family members, and friends.
2) *Attribute-based access:* Attribute-based encryption (ABE) is used for fine-grained access control in the healthcare IoT network. The family members and data users, such as clinicians, researchers, and insurers, are assigned secret keys bound to their respective sets of attributes and, thus, have different data access privileges.
3) *Break-glass access:* LiBAC realizes password-based break-glass access. A patient presets a password and preshares it with a set of emergency contact persons (ECPs). In an emergency situation, an ECP utilizes the password to extract the break-glass key and decrypts the medical files. This approach avoids abuse of break-glass authority because only ECPs who know the password are capable of extracting the break-glass key.
4) *Lightweight construction:* In order to reduce computation and storage overhead in wireless healthcare IoTs, LiBAC is designed to be lightweight in data transmission and computations.

A. Related Work

1) ABE for Fine-Grained Access Control: In 2007, Ostrovsky *et al.* [8] proposed an ABE scheme with nonmonotonic access structure, which is proved secure based on decisional bilinear Diffie–Hellman (DBDH) assumption. In order to reduce the decryption overhead, Lai *et al.* [9] proposed to outsource a significant portion of decryption operation to a public server and presented an ABE scheme with verifiable outsourced decryption. The similar problem is also studied in [10]. If the malicious user intentionally sells his decryption key for profit, it is necessary to identity the traitor's identity and deprive his decryption privilege. This traitor tracing problem was investigated in [11]. Zhou *et al.* [12] put forth a multiauthority ABE with white-box traceability for e-health systems. Rouselakis *et al.* [13] proposed a large-universe multiauthority ABE to reduce the size of the public parameter. Yang *et al.* [14] designed a lightweight ABE for health IoT in distributed environment. Deng *et al.* [15] suggested a hierarchical ABE with short ciphertext. Luo *et al.* [16] proposed a hierarchical ABE with multiple authorities for the mobile social network. The hierarchical ABE was also investigated in [17]–[19].

2) Break-Glass Access Control: In 2009, Brucker *et al.* [20] proposed a generic break-glass access control model based on the notion of emergency levels, in which several emergency levels were designed to classify various kinds of violations to the regular policy. Later, Brucker *et al.* [21] integrated the break-glass concept into the ABE security architecture, in which a hierarchy of emergency attributes were utilized and deemed as controlled overriding of the ordinary access control constrains. Marinovic *et al.* [22] put forth a new break-glass access control model named Rumpole. It leveraged a declarative request language in the break-glass decision process, and required a policy writer to restrict the time and mode to enable the break-glass access. A generic process-related role-based access control model with break-glass was suggested by Schefer *et al.* [23]. In 2016, Maw *et al.* [24] proposed a break-glass access control model for a wireless sensor network, which was also based on the role model. It could detect the access policy violations from the users. As [20]–[24] only gave out the break-glass access control model without concrete construction, Zhang *et al.* [25] put forth a password-controlled break-glass access control scheme, which was constructed based on identity-based encryption and could not exert fine-grained access control on the shared ciphertexts in the normal situation.

II. Preliminary

A. Bilinear Pairing and Hardness Assumption

Let $\mathbb{G}$ and $\mathbb{G}_T$ be cyclic groups and g_1 be a generator of $\mathbb{G}$. The bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ has the following characteristics:

1) *bilinear:* $\forall h_1, h_2 \in \mathbb{G}$ and $\forall a, b \in \mathbb{Z}_p$, $e(h_1^a, h_2^b) = e(h_1, h_2)^{ab}$;
2) *nondegenerate:* $e(g_1, g_1) \neq 1$; and
3) *computable:* $\forall h_1, h_2 \in \mathbb{G}$, $e(h_1, h_2)$ can be efficiently calculated.

DBDH assumption: Let $a, b, c \in \mathbb{Z}_p$ and $T \in \mathbb{G}_T$ be random numbers, and g_1 be a generator of $\mathbb{G}$. Given a tuple $\vec{y} = (g_1, g_1^a, g_1^b, g_1^c)$, no probabilistic polynomial-time algorithm $\mathcal{A}$ can distinguish $e(g_1, g_1)^{abc}$ from T with nonnegligible advantage. The advantage ϵ of $\mathcal{A}$ is defined as $|Pr[\mathcal{A}(\vec{y}, e(g_1, g_1)^{abc}) = 0] - Pr[\mathcal{A}(\vec{y}, T) = 0]|$.

B. Linear Secret Sharing Scheme

Definition 1 (Linear Secret Sharing Scheme (LSSS) [26]): A secret-sharing scheme Π over a set of parties $\mathcal{P}$ is called linear (over $\mathbb{Z}_p$) if

1) The shares for each party form a vector over $\mathbb{Z}_p$.
2) There exists a matrix $\mathbb{A}$ with l rows and n columns called the share-generating matrix for Π. For all $i = 1, \ldots, l$, the ith row $\mathbb{A}_i$ of the matrix $\mathbb{A}$ is labeled by a party $\rho(i)$ (ρ is a function from $\{1, \ldots, l\}$ to $\mathcal{P}$). Set the column vector $\mathbf{v} = (z, \lambda_2, \ldots, \lambda_n)$, where $z \in \mathbb{Z}_p$ is the secret to be shared and $\lambda_2, \ldots, \lambda_n \in \mathbb{Z}_p$ are randomly chosen. $\mathbb{A}\mathbf{v}$ is the vector of l shares of the secret z according to Π and the share $\mathbb{A}_i\mathbf{v}$ belongs to party $\rho(i)$.

Suppose that Π is an LSSS for the access structure Φ. Let $S \in \Phi$ be any authorized set and $I \subset \{1, \ldots, l\}$ be defined as $I = \{i : \rho(i) \in S\}$. Then, there exists constants $\{\omega_i \in \mathbb{Z}_p\}_{i \in I}$

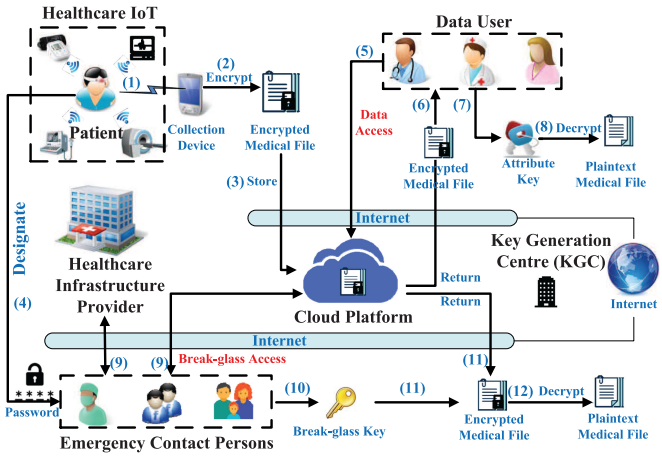


Fig. 1. System architecture.

such that, if $\{z_i\}_{i\in I}$ are valid shares of any secret z according to Π, then $\sum_{i\in I}\omega_i z_i = z$ and $\sum_{i\in I}\omega_i \mathbb{A}_i = (1, 0, \ldots, 0)$. Furthermore, it is shown in [26] that these constants $\{\omega_i\}_{i\in I}$ can be found in time polynomial in the size of the share-generating matrix $\mathbb{A}$. For unauthorized sets, no such constants exist.

III. System Architecture and Its Model

A. System Architecture

The architecture of LiBAC consists of a key generation center (KGC), a cloud platform (CP), a healthcare infrastructure provider (HIP), the patient (PA), the data users, and the ECP. These entities and the interactions between them are shown in Fig. 1 and introduced in the following.

KGC generates master public/secret key pair for the whole system and attribute secret keys for the patients and the data users.

CP has plenty storage and calculation capability, and provides outsourced storage and computation service to its customers.

HIP can be the hospital or other medical institutions, who provides IoT infrastructure to the patients. The medical devices (such as electrocardiograph machine, B-ultrasound instrument, electronic sphygmomanometer, and CT scanner) of the HIP are connected to the Internet to form a healthcareIoT.

PA gets medical services from the HIP. The healthcare IoT collects the physiological data and medical images of the patient, which are transmitted to a collection device through a wireless network (see step 1 in Fig. 1). In order to protect the sensitive medical data and realize fine-grained access control, the patient specifies an access policy, which is utilized to encrypt the medical file (see step 2 in Fig. 1). Then, the encrypted medical file is stored in CP via Internet (see step 3 in Fig. 1).

Considering the emergency situations, the patient generates a password-based break-glass key such that all encrypted files of the patient can be decrypted by the break-glass key. The patient designates a list of ECPs to preshare the password with (see step 4 in Fig. 1), which can be utilized to recover the break-glass key.

Data users can be physicians, nurses, and other medical staff in the HIP, or the friends and relatives of the patients. Each of them registers with the KGC and receives the attribute secret key according to their characteristics. They are able to access to patient's data if their attributes satisfy the access policies (see step 5 in Fig. 1). Receiving the encrypted medical files from the CP (see step 6 in Fig. 1), they utilize the attribute secret key (see step 7 in Fig. 1) to decrypt and obtain the plaintext medical files (see step 8 in Fig. 1).

ECPs are designated by the patient and know the patient's password. In an emergency situation, they interact with the CP and HIP (see step 9 in Fig. 1) to recover the break-glass key (see step 10 in Fig. 1). ECPs query a patient's encrypted files from CP (see step 11 in Fig. 1) and utilizes the break-glass key to recover the patient's medical records (see step 12 in Fig. 1).

B. Security Model

To ensure the security of the ABE encrypted medical files, the system should be indistinguishable against chosen plaintext attack (IND-CPA). The concrete security model can be found in [9] and [10], in which an adversary cannot distinguish two challenge ciphertexts given the two underlying plaintexts. The IND-CPA security has two security levels: selective secure and fully secure. In the selective security model, the adversary has to output the challenge access policy before the system is set up. In the fully security model, the adversary designates the challenge access policy in the challenge phase rather than the very beginning. It is obvious that the fully security model has higher security level than the selective model. LiBAC is secure in the fully security model.

To ensure the security of the break-glass access control mechanism, CP and HIP cannot collude. Patient's password and break-glass key can be used to decrypt all of the patient's encrypted healthcare files, and should be protected in the following two aspects:

1) The CP and HIP cannot deduce patient's password or the break-glass key from the auxiliary information that are stored in CP and HIP.
2) In the break-glass key extraction process, ECP sends an encapsulated password to CP and HIP. The system guarantees that CP and HIP still cannot get the password or break-glass key when they obtain the encapsulated password.

IV. System Workflow

In this section, we briefly introduce the system workflow that is different from the traditional ABE based access control proposals. The break-glass key generation and extraction processes as well as two types of access control mechanisms are illustrated.

A. Break-Glass Key Generation

Fig. 2 shows the break-glass key generation procedure. The patient sets a password pw for his encrypted files (see step 1 in Fig. 2), designates a ECP list (see step 2 in Fig. 2) and preshares the password with the ECPs in the list (see step 3 in Fig. 2). The ECP list is stored in HIP.

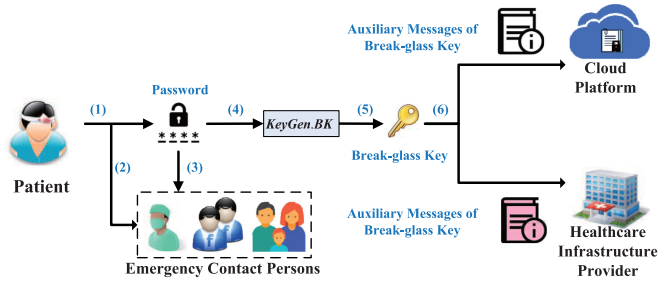


Fig. 2. Break-glass key generation.

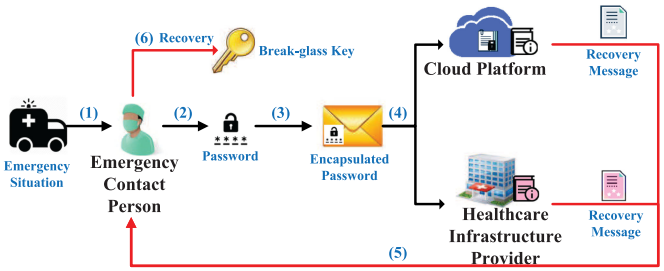


Fig. 3. Break-glass key extraction.

Then, the patient runs break-glass key generation algorithm $KeyGen.BK$ (see step 4 in Fig. 2) to derive the break-glass key BK from the password pw (see step 5 in Fig. 2). Two auxiliary messages $(\mathrm{bk}_1, \mathrm{bk}_2)$ of the break-glass key are also generated, which are the clues to recover the break-glass key BK from the password pw. The auxiliary messages $(\mathrm{bk}_1, \mathrm{bk}_2)$ of the break-glass key are secretly stored by CP and HIP, respectively (see step 6 in Fig. 2).

B. Break-Glass Key Extraction

Fig. 3 presents the break-glass key extraction procedure. If the patient encounters emergency situations (see step 1 in Fig. 3), ECP of the patient utilizes the password pw to extract the break-glass key BK.

In order to protect the password, ECP utilizes the password pw (see step 2 in Fig. 3) as input to generate an encapsulated password (see step 3 in Fig. 3), which is sent to CP and HIP (see step 4 in Fig. 3). Both the CP and HIP cannot deduce the password pw from the encapsulated password. CP (resp. HIP), with the auxiliary messages bk_1 (resp. bk_2) of the break-glass key and the encapsulated password as input, computes and sends an auxiliary recovery message Ψ_1 (resp. Ψ_2) to the ECP (see step 5 in Fig. 3). Receiving these auxiliary recovery messages Ψ_1 and Ψ_2 from CP and HIP, respectively, ECP recovers the break-glass key BK (see step 6 in Fig. 3), which can decrypt all the encrypted medical files of the patient.

C. Two Access Control Mechanisms

As shown in Fig. 4, LiBAC provides two access control mechanisms for attribute-based access and break-glass access and the following algorithms are involved: delegation key generation algorithm $KeyGen.Del$, partial decryption algorithm

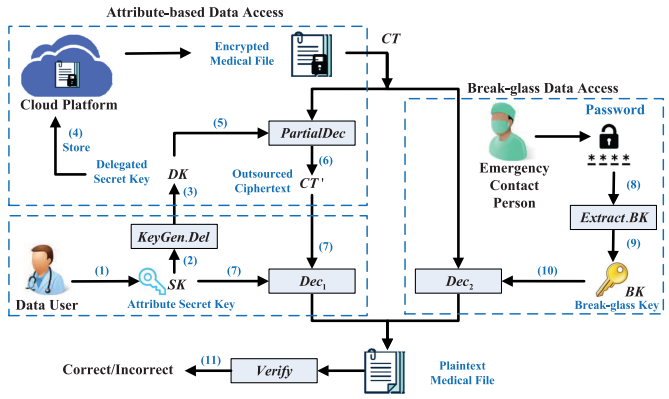


Fig. 4. Two access control mechanisms.

$PDec$, break-glass key extraction algorithm $Extract.BK$, type-1 decryption algorithm Dec_1, and type-2 decryption algorithm Dec_2. These algorithms are detailed in Section V.

For the attribute based access, a data user utilizes his attribute secret key SK to decrypt the ciphertext (see step 1 in Fig. 4). In LiBAC, the outsourced decryption technique in [9] and [10] is utilized to alleviate the data user's decryption burden. The data user inputs SK (see step 2 in Fig. 4) into the delegation key generation algorithm $KeyGen.Del$ to generate a delegation key DK (see step 3 in Fig. 4), which is sent to the CP in an authenticated manner (see step 4 in Fig. 4). In the data access process, the CP partially decrypt the encrypted medical file on the data user's behalf. Using the the delegation key DK, CP runs the partial decryption algorithm $PDec$ (see step 5 in Fig. 4) to transform the ciphertext CT into CT$'$ (see step 6 in Fig. 4). Utilizing the attribute secret key SK, the data user runs type-1 decryption algorithm Dec_1 to recover the plaintext medical file (see step 7 in Fig. 4). In this process, the data user only need to perform one exponential calculation to recover the message from CT$'$.

For the break-glass access, ECP utilizes the password pw (see step 8 in Fig. 4) and the break-glass key extraction algorithm $Extract.BK$ to recover the break-glass key BK (see step 9 in Fig. 4). Then, ECP runs type-2 decryption algorithm Dec_2 to obtain the plaintext medical file (see step 10 in Fig. 4).

After the decryption algorithms, a verification method is provided to verify whether the recovered medical file is correct (see step 11 in Fig. 4). The misbehavior of the CP or HIP, such as the CP sends to the data user an incorrect CT$'$, or the CP (or HIP) sends to ECP an incorrect auxiliary recovery message, can be detected.

V. Proposed System

In this section, we present a concrete LiBAC for the healthcare IoT. The notation $a \in_R S$ represents that an element a is randomly selected from a set S.

A. System Setup

Taken as input the security parameter 1^κ, the KGC generates the master public/secret keys MPK/MSK for the system.

$Setup(1^\kappa) \rightarrow (MPK, MSK)$*:* KGC chooses hash functions $H_1 : \{0,1\}^* \rightarrow \mathbb{Z}_p^*$, $H_2 : \{0,1\}^* \rightarrow \mathbb{G}_T$, $H_3 : \{0,1\}^* \rightarrow \mathcal{K}$ and cryptographically secure symmetric encryption and decryption pair $SEnc/SDec$ with secret key space $\mathcal{K}$. Select $\alpha, \beta \in_R \mathbb{Z}_p^*$, $g_1 \in_R \mathbb{G}$ and compute $g_2 = g_1^\beta, Y = e(g_1, g_1)^\alpha$. Set $MPK = (g_1, g_2, Y)$ and $MSK = (\alpha, \beta)$. MPK is a default input in the following algorithms.

B. Key Generation for User

Each user is assigned a set of attributes S based on his/her position in the healthcare system and based on the set of attributes, KGC generates the attribute secret key SK for the user. Note that a user can be a patient or a data user.

$KeyGen.User(MSK, S) \rightarrow SK$*:* Suppose the user's attribute set $S = (\text{attr}_1, \text{attr}_2, \ldots, \text{attr}_k)$. Compute $\sigma_i = H_1(\text{attr}_i) \in Z_p^*$ for $1 \le i \le k$. Select $u, r_1, r_2, \tau \in_R \mathbb{Z}_p^*$ and compute $\text{sk}_1 = g_1^{\alpha - u}$, $\text{sk}_{2,i} = g_1^{u(\sigma_i + \beta)^{-1}}$, $\text{sk}_3 = r_1$, $\text{sk}_4 = r_2$, $\text{sk}_5 = \tau$. Set $\text{SK} = (\text{sk}_1, \{\text{sk}_{2,i}\}_{i \in [k]}, \text{sk}_3, \text{sk}_4, \text{sk}_5)$.

C. Delegation Key Generation

A user generates delegation key DK from the attribute secret key SK. Then, DK is sent to the CP in authenticated manner.

$KeyGen.Del(SK) \rightarrow DK$*:* The user calculates $\text{dk}_1 = \text{sk}_1^{\text{sk}_5}$, $\text{dk}_{2,i} = \text{sk}_{2,i}^{\text{sk}_5}$, $\text{dk}_3 = g_1^{\text{sk}_3}$, $\text{dk}_4 = g_2^{\text{sk}_4}$, and sets $\text{DK} = (\text{dk}_1, \{\text{dk}_{2,i}\}_{i \in [k]}, \text{dk}_3, \text{dk}_4)$.

D. Password-Based Break-Glass Key Generation

A patient (with identity ID_{PA}) selects a password pw and generates a break-glass key BK based on pw. At the same time, the auxiliary information $(\text{bk}_1, \text{bk}_2)$ of break-glass key are generated, which are used to recover the BK from the password pw. Then, bk_1 and bk_2 are sent to CP and HIP, respectively. The patient designates a list of ECPs and secretly preshares his/her password pw with the ECPs. The ECP list is stored in HIP.

$KeyGen.BK(pw) \rightarrow (BK, bk_1, bk_2)$. The patient computes $\zeta = H_1(\text{ID}_{\text{PA}}, \text{pw})$ and chooses random $\zeta_1, \zeta_2 \in_R \mathbb{Z}_p^*$ such that $\zeta = \zeta_1 + \zeta_2$. Selects $K, K_1 \in_R G$ and sets the break-glass key as $\text{BK} = K$.

CP randomly selects $\theta_1 \in_R \mathbb{Z}_p^*$, calculates $P_{\text{CP}} = g_1^{\theta_1}$ and sends P_{CP} to the patient. HIP randomly selects $\theta_2 \in_R \mathbb{Z}_p^*$, calculates $P_{\text{HIP}} = g_1^{\theta_2}$, and sends P_{HIP} to the patient. Receiving $(P_{\text{CP}}, P_{\text{HIP}})$, the patient selects $r \in_R \mathbb{Z}_p^*$ and calculates

$$K_2 = K \cdot (K_1)^{-1} \cdot (P_{\text{CP}} \cdot P_{\text{HIP}})^r$$
$$\Lambda_1 = (P_{\text{HIP}})^{-2\zeta_1} \cdot g_1^{-r}, \quad \Lambda_2 = (P_{\text{CP}})^{-2\zeta_2} \cdot g_1^{-r}.$$

Set break-glass key auxiliary messages as $\text{bk}_1 = (K_1, \Lambda_1), \text{bk}_2 = (K_2, \Lambda_2)$, which are sent to CP and HIP, respectively. CP securely stores $(\text{bk}_1, P_{\text{CP}}, \theta_1)$ and HIP securely stores $(\text{bk}_2, P_{\text{HIP}}, \theta_2)$.

E. Password-Based Break-Glass Key Extraction

An ECP of the patient interacts with the CP and HIP to extract the break-glass key BK using the password pw.

$Extract.BK(pw, bk_1, bk_2) \rightarrow BK$*:* Receiving the break-glass key extraction request, CP and HIP, respectively, sends $P_{\text{CP}}, P_{\text{HIP}}$ to ECP. Then, ECP chooses $s \in_R \mathbb{Z}_p^*$ and calculates $\zeta = H_1(\text{ID}_{\text{PA}}, \text{pw})$, $\Gamma_1 = (P_{\text{HIP}})^\zeta \cdot g_1^s$, $\Gamma_2 = (P_{\text{CP}})^\zeta \cdot g_1^s$. The elements (Γ_1, Γ_2) are sent to CP and HIP, respectively. CP computes $\Psi_1 = K_1 \cdot (\Lambda_1 \cdot \Gamma_1)^{\theta_1}$, and HIP computes $\Psi_2 = K_2 \cdot (\Lambda_2 \cdot \Gamma_2)^{\theta_2}$, which are sent to the ECP. Then, ECP recovers the break-glass key by computing $\text{BK} = (\Psi_1 \cdot \Psi_2) \cdot (P_{\text{CP}} \cdot P_{\text{HIP}})^{-s}$.

F. Encryption

A patient uses an access policy $(\mathbb{A}, \rho)$ and the break-glass key BK to encrypt a medical file M (with file identity FID), where $\mathbb{A} \in \mathbb{Z}_p^{l \times n}$, ρ maps $\mathbb{A}$'s rows to attributes.

$Enc(M, (\mathbb{A}, \rho), SK_{PA}, BK) \rightarrow \text{CT}$*:* The patient chooses $z, \lambda_2, \ldots, \lambda_n \in_R \mathbb{Z}_p^*$ and sets $\mathbf{v} = (z, \lambda_2, \ldots, \lambda_n)^\top$. Compute $z_i = \mathbb{A}_i \mathbf{v}$ for $i \in [l]$. The ciphertext CT is constructed as $\Upsilon = H_2(\text{BK}, \text{ID}_{\text{PA}}, FID)$, $C_0 = \Upsilon \cdot Y^z$, $C_1 = g_1^z$, $C_{2,i} = \rho(i) z_i / r_1$, $C_{3,i} = z_i / r_2$, $C_M = SEnc(H_3(\Upsilon), M || 0^\varpi)$, where $M||0^\varpi$ denotes ϖ-0s are concatenated after M. Set $\text{CT} = (C_M, C_0, C_1, \{C_{2,i}, C_{3,i}\}_{i \in [l]})$ and outsource $(\text{CT}, (\mathbb{A}, \rho))$ to CP for remote storage.

G. Partial Decryption

Receiving the data access request from a data user, CP verifies whether the data user's attribute set satisfies the access policy of the ciphertext. If it does not, the request is rejected. Otherwise, CP utilizes the delegation key DK of the data user to partially decrypt the ciphertext. The purpose of the partial decryption is to reduce the data user's decryption burden.

1) $PDec(CT, DK) \rightarrow CT'$*:* CP utilizes the LSSS mechanism (introduced in Section II-B) to find a set of constants $\{\omega_i \in \mathbb{Z}_p\}_{i \in [l]}$ such that $\sum_i \omega_i \mathbb{A}_i = (1, 0, \ldots, 0)$. Then, CP calculates

$$\Omega = e(C_1, \text{dk}_1) e\left(\text{dk}_3, \prod_{i \in I} \text{dk}_{2,i}^{C_{2,i} \cdot \omega_i}\right) e\left(\text{dk}_4, \prod_{i \in I} \text{dk}_{2,i}^{C_{3,i} \cdot \omega_i}\right)$$
$$= e(g_1, g_1)^{\alpha \cdot z \cdot \tau}$$

and sends the transformed ciphertext $\text{CT}' = (C_M, C_0, \Omega)$ to data user.

H. Decryption With Attribute Key and Verification

In the normal situation, a data user utilizes his attribute secret key SK and type-1 decryption algorithm Dec_1 to recover the plaintext, and then verifies its correctness.

$Dec_1(CT', SK) \rightarrow M/\perp$*:* The data user computes $\Upsilon = C_0 \cdot \Omega^{-\text{sk}_5}$ and $M' = SDec(H_3(\Upsilon), C_M)$. If $M' = M||0^\varpi$, it means that the transformed ciphertext CT′ is correct and the recovered plaintext medical file is M. Otherwise, it outputs $\perp$.

I. Decryption With Break-Glass Key and Verification

In an emergency situation, ECP utilizes the break-glass key BK and type-2 decryption algorithm Dec_2 to decrypt the ciphertext.

$Dec_2(CT, BK) \rightarrow M/\perp$: ECP computes $\Upsilon = H_2(\text{BK}, \text{ID}_{\text{PA}}, \text{FID})$ and $M' = SDec(H_3(\Upsilon), C_M)$. If $M' = M||0^{\varpi}$, it means that the break-glass key BK is correctly extracted and the recovered plaintext medical file is M. Otherwise, it outputs $\perp$.

VI. Security Analysis

Theorem 1: Assume that the DBDH assumption holds, then LiBAC is fully IND-CPA secure.

Proof: Suppose there is an adversary $\mathcal{A}$ that can distinguish different ciphertexts. Then, we can build an algorithm $\mathcal{C}$ to break the DBDH assumption, who is given the DBDH tuple $\vec{y} = (g_1, g_1^a, g_1^b, g_1^c)$.

1) *Setup:* The challenger $\mathcal{C}$ selects $\alpha', \beta \in_R \mathbb{Z}_p^*$ and implicitly sets $\alpha = \alpha' + ab$. Compute $Y = e(g_1^a, g_1^b)e(g_1, g_1)^{\alpha'} = e(g_1, g_1)^{\alpha}$ and $g_2 = g_1^{\beta}$. $\mathcal{C}$ sends the master public key $\text{MPK} = (g_1, g_2, Y)$ to $\mathcal{A}$.
2) *Phase 1:* $\mathcal{A}$ adaptively issues the following queries:
   a) $\mathcal{O}_{\text{SK}}$: $\mathcal{A}$ queries on the attribute secret key of attribute set S (with $|S| = k$). $\mathcal{C}$ chooses $r_1, r_2, \tau, u' \in_R \mathbb{Z}_p^*$ and implicitly sets $u = ab + u'b$, $\mathcal{C}$ picks $k_i \in_R \mathbb{Z}_p^*$ such that k_i is invertible in $\mathbb{Z}_p^*$ for $i \in [k]$. Implicitly set $\sigma_i = b \cdot k_i^{-1} - \beta$ and constructs $\text{sk}_3 = r_1, \text{sk}_4 = r_2, \text{sk}_5 = \tau$,

$$\begin{aligned}
\text{sk}_1 &= g_1^{\alpha - u} = g_1^{(\alpha' + ab) - (ab + u'b)} \\
&= g_1^{\alpha' - u'b} = g_1^{\alpha'} \cdot (g_1^b)^{u'} \\
\text{sk}_{2,i} &= g_1^{u(\sigma_i + \beta)^{-1}} = g_1^{(ab + u'b)(b \cdot k_i^{-1} - \beta + \beta)^{-1}} \\
&= g_1^{(ab + u'b)(k_i \cdot b^{-1})} = g_1^{k_i(a + u')} \\
&= (g_1^a)^{k_i} \cdot g_1^{k_i \cdot u'}
\end{aligned}$$

   The secret key $\text{SK} = (\text{sk}_1, \{\text{sk}_{2,i}\}_{i \in [k]}, \text{sk}_3, \text{sk}_4, \text{sk}_5)$ is sent to $\mathcal{A}$.
   b) $\mathcal{O}_{\text{DK}}$: $\mathcal{A}$ queries on the delegation key of data user with attribute set S. $\mathcal{C}$ runs $\mathcal{O}_{\text{SK}}$ to construct the attribute secret key SK and computes $\text{dk}_1 = \text{sk}_1^{\text{sk}_5}$, $\text{dk}_{2,i} = \text{sk}_{2,i}^{\text{sk}_5}$, $\text{dk}_3 = g_1^{\text{sk}_3}$, $\text{dk}_4 = g_2^{\text{sk}_4}$. The constructed delegation key $\text{DK} = (\text{dk}_1, \{\text{dk}_{2,i}\}_{i \in [k]}, \text{dk}_3, \text{dk}_4)$ is sent to $\mathcal{A}$.
   c) $\mathcal{O}_{PDec}$: $\mathcal{A}$ issues partial decryption query on ciphertext CT for data user with attribute set S. $\mathcal{C}$ runs $\mathcal{O}_{\text{DK}}$ to construct the delegation key DK. $\mathcal{C}$ computes $\Omega = e(C_1, \text{dk}_1)e(\text{dk}_3, \prod_{i \in I} \text{dk}_{2,i}^{C_{2,i} \cdot \omega_i})$ $e(\text{dk}_4, \prod_{i \in I} \text{dk}_{2,i}^{C_{3,i} \cdot \omega_i})$. Then, $\text{CT}' = (C_M, C_0, \Omega)$ is sent to $\mathcal{A}$.
   d) $\mathcal{O}_{Dec_1}$: $\mathcal{A}$ issues type-1 decryption query on ciphertext CT for data user with attribute set S. $\mathcal{C}$ runs $\mathcal{O}_{PDec}$ to construct the transformed ciphertext CT′ and $\mathcal{O}_{\text{SK}}$ to construct the attribute secret key SK. $\mathcal{C}$ computes $\Upsilon = C_0 \cdot \Omega^{-\text{sk}_5}$ and $SDec(H_3(\Upsilon), C_M)$ to recover the message M.
3) *Challenge:* $\mathcal{A}$ sends an access policy $(\mathbb{A}^*, \rho^*)$, a message M^* and two challenge data $(\Upsilon_0^*, \Upsilon_1^*)$ to $\mathcal{C}$, where $\mathbb{A}^* \in \mathbb{Z}_p^{l^* \times n^*}$ and ρ^* maps $\mathbb{A}^*$'s rows to attributes. The constraint is that the secret attribute key of attribute set S that satisfies $(\mathbb{A}^*, \rho^*)$ is not queried in phase 1. $\mathcal{C}$ selects $z_i^* \in_R \mathbb{Z}_p^*$ for $1 \le i \le l^*$. $\mathcal{C}$ tosses a coin $\mu \in \{0, 1\}$ and computes $C_0^* = \Upsilon_{\mu}^* \cdot T \cdot e(g^c, g^{\alpha'}), C_1^* = g_1^c, C_{2,i}^* = \rho(i) z_i^* / r_1, C_{3,i}^* = z_i^* / r_2$. Set $k_{SE}^* = H_3(\Upsilon_{\mu}^*)$. Compute $C_{M^*} = SEnc(k_{SE}^*, M^*||0^{\varpi})$, where $||$ denotes concatenation of a string. The challenge ciphertext $\text{CT}^* = (C_{M^*}, C_0^*, C_1^*, \{C_{2,i}^*, C_{3,i}^*\}_{i \in [l]})$ is sent to $\mathcal{A}$.
4) *Phase 2:* It is the same as *Phase 1* with the constraint that S does not satisfy $(\mathbb{A}^*, \rho^*)$.
5) *Guess:* $\mathcal{A}$ outputs a guess $\mu' \in \{0, 1\}$. If $\mu' = \mu$, $\mathcal{C}$ outputs 1 indicating $T = e(g_1, g_1)^{abc}$. Otherwise, it outputs 0 indicating T is random. ■

Next, we analyze the security of break-glass access control.

1) The CP (resp. HIP) is unable to get patient's pw or BK from the auxiliary information $\text{bk}_1 = (K_1, \Lambda_1)$ (resp. $\text{bk}_2 = (K_2, \Lambda_2)$). Since $\Lambda_1 = (P_{\text{HIP}})^{-2\zeta_1} \cdot g_1^{-r}$, CP cannot deduce ζ_1 from Λ_1 because r is an unknown random number. CP are not capable to deduce $\text{BK} = K$ from K_1 either. The security analysis of HIP is similar as CP.
2) In the break-glass key extraction phase, the CP (resp. HIP) is unable to get patient's pw or BK from the encapsulated password Γ_1 (resp. Γ_2). Since $\Gamma_1 = (P_{\text{HIP}})^{\zeta} \cdot g_1^s$ and $\zeta = H_1(\text{ID}_{\text{PA}}, \text{pw})$, CP cannot deduce ζ from Γ_1 because s is an unknown random number. The security analysis of HIP is similar as CP.

TABLE I
Communication Overhead Comparison

Parameter	LiBAC	[8]	[11]
|MPK|	$\|\mathbb{G}\| + \|\mathbb{G}_T\|$	$(2U+3)\|\mathbb{G}\|$	$6\|\mathbb{G}\| + \|\mathbb{G}_T\|$
|SK|	$(\|S\|+1)\|\mathbb{G}\| + 3\|\mathbb{Z}_p\|$	$(5\|S\|)\|\mathbb{G}\|$	$(2\|S\|+3)\|\mathbb{G}\| + \|\mathbb{Z}_p\|$
|CT|	$\|\mathbb{G}\| + \|\mathbb{G}_T\| + (2l)\|\mathbb{Z}_p\|$	$(2l+1)\|\mathbb{G}\| + \|\mathbb{G}_T\|$	$(3l+2)\|\mathbb{G}\| + \|\mathbb{G}_T\|$
|BK|	$\|\mathbb{G}\|$	$\perp$	$\perp$
$\|\text{bk}_1\|/\|\text{bk}_2\|$	$2\|\mathbb{G}\|$	$\perp$	$\perp$

VII. Performance Analysis

We compare LiBAC with the existing ABE schemes [8], [11] in terms of communication and computation overheads. We implement these schemes using the pairing based based crypto (PBC) library [27] to evaluate their efficiencies. The type-A elliptic curve is selected. The experiments are conducted on PC with Intel(R) Core(TM) i5-6200U CPU @ 2.3 GHz, 4 GB RAM and running Windows 10 64-bit operation system.

Table I and Fig. 5(a)–(c) compare the communication overhead of these schemes. The notations $|\mathbb{G}|$, $|\mathbb{G}_T|$, and $|\mathbb{Z}_p|$ represent the bit length of the element in $\mathbb{G}$, $\mathbb{G}_T$, and $\mathbb{Z}_p$, respectively. The notations $|MPK|$, |SK|, and |CT| denote the sizes of the master public key MPK, user's attribute secret key SK and ciphertext CT, respectively. $|U|$ represent the size of the universe attribute set. l is the row number of the access policy matrix $\mathbb{A}$ and $|S|$ is the size of user's attribute set. For type-A elliptic

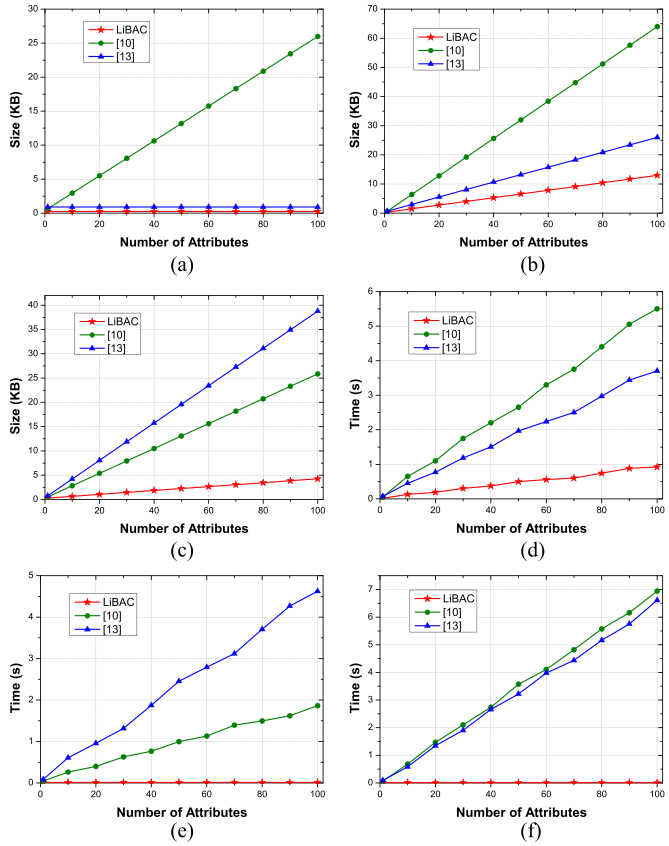


Fig. 5. Communication and computation efficiency. (a) Master public key size. (b) Secret key size. (c) Ciphertext size. (d) *KeyGen:User* time. (e) Encryption time. (f) Dec_1 time.

curve, the bit length of an element in $\mathbb{G}$, $\mathbb{G}_T$, and $\mathbb{Z}_p$ are 1024, 1024, and 160 bit, respectively.

1) *Master public key size:* Fig. 5(a) shows the master public key size of the three schemes. LiBAC is a large universe construction since the master public key size is a constant and has no constraint on the size of the universe attribute set. The advantage of large universe construction is that new attribute can be added after the system is set up. Another merit is that the storage space is small for resource-limited IoT devices. The MPK in LiBAC consists of one element in group $\mathbb{G}$ and one element in group $\mathbb{G}_T$, and the bit length of MPK is 0.256 KB. The size of MPK in scheme [11] is also a constant and has 0.896 KB, which is two times larger than LiBAC. The master public key in scheme [8] linearly grows with the size $|U|$ of the universe attribute set. When $|U| = 100$, the size of MPK is 25.984 KB in scheme [8], which is 100.5 times larger than LiBAC.
2) *Attribute secret key size:* Fig. 5(b) shows the attribute secret key size of the users. It is obvious that LiBAC has the least size. When the size of attribute set grows to 100, the sizes of SK are 64, 26.004, and 12.988 KB in [8], [11] and LiBAC, respectively. The sizes of SK in [8], [11] are 3.9 and 1 times larger than LiBAC when $|S| = 100$, respectively.

TABLE II
COMPUTATION OVERHEAD COMPARISON

Algorithm	LiBAC	[8]	[11]
$KeyGen.User$	$(\|S\|+1)t_{e_1}$	$(6\|S\|)t_{e_1}$	$(4\|S\|+4)t_{e_1}$
Enc	$t_{e_1}+t_{e_2}$	$t_p+t_{e_2}+(2l+1)t_{e_1}$	$t_p+t_{e_2}+(5l+2)t_{e_1}$
Dec_1	t_{e_1}	$3\|S\|t_p+\|S\|t_{e_1}+2\|S\|t_{e_2}$	$(3\|S\|+1)t_p+(\|S\|+1)t_{e_1}+\|S\|t_{e_2}$
$PDec$	$3t_p+(2l)t_{e_1}$	$\perp$	$\perp$
$KeyGen.$BK	$6t_{e_1}$	$\perp$	$\perp$
$Extract.$BK	$6t_{e_1}$	$\perp$	$\perp$

3) *Ciphertext size:* Fig. 5(c) shows the ciphertext size. LiBAC also has the least ciphertext size. When the size of attribute set grows to 100, the sizes of CT are 25.856, 38.784, and 4.256 KB in [8], [11] and LiBAC, respectively. The sizes of CT in [8] and [11] are five and eight times larger than LiBAC when $|S| = 100$, respectively.
4) *Break-glass key and its auxiliary information size:* The schemes in [8] and [11] do not have break-glass access function nor break-glass key. The sizes of BK, bk_1, and bk_2 are 0.128, 0.256, and 0.256 KB in LiBAC, respectively.

Table II and Fig. 5(d)–(f) compare the computation overhead of these schemes. Let t_{e_1}, t_{e_2}, and t_p denote the operation times of a exponentiation computation in $\mathbb{G}$, a exponentiation computation in $\mathbb{G}_T$ and a pairing computation, respectively.

1) $KeyGen.User$*:* Fig. 5(d) shows the computation efficiency of $KeyGen.User$ algorithm of these schemes. LiBAC has the least computation cost. When $|S| = 100$, the running times of $KeyGen.User$ are 5.504, 3.706, and 0.926 s in [8], [11], and LiBAC, respectively. The running times of $KeyGen.User$ in [8] and [11] are 4.9 and 3 times larger than LiBAC when $|S| = 100$, respectively.
2) Enc*:* Fig. 5(e) shows the computation efficiency of Enc algorithm of these schemes. When $|S| = 100$, the running times of Enc are 1.864, 4.625, and 0.012 s in [8], [11], and LiBAC, respectively. LiBAC has the far less execution time for Enc algorithm. The running times of Enc in [8] and [11] are 154 and 384 times larger than LiBAC when $|S| = 100$, respectively.
3) Dec_1*:* Fig. 5(f) shows the computation efficiency of Dec_1 algorithm of these schemes. When $|S| = 100$, the running times of Dec_1 are 6.842, 6.611, and 0.009 s in [8], [11], and LiBAC, respectively. The running times of Dec_1 in [8] and [11] are 759 and 733 times larger than LiBAC when $|S| = 100$, respectively.
4) $KeyGen.BK$, $Extract.BK$, and Dec_2*:* In LiBAC, the break-glass key generation and extraction processes are lightweight, which take only six exponentiation computations in group $\mathbb{G}$ and the execution time is merely 0.055 s. The ECP can utilizes two hash function to recover the symmetric key of the ciphertext and the execution time is 0.021 s.

To conclude, LiBAC incurs much smaller communication and computation costs compared with other schemes. This

lightweight system is applicable for the resource-constrained devices in healthcare IoT network.

VIII. Conclusion

In this paper, we proposed a lightweight data sharing system for healthcare IoT with two access control modes. In attribute-based access mode, authorized data users use their attribute secret keys to decrypt and access patient's medical records. In break-glass access mode, a patient preshares a password with a set of ECPs and the password is the clue to recover the break-glass key. In emergency circumstance, an ECP utilizes the password to extract the break-glass key, and decrypts the medical records to save patient's life. Based on the DBDH assumption, encrypted medical records are secure in the sense that they are indistinguishable against chosen plaintext attacks. The break-glass key generation and extraction algorithms leak no information about the password and break-glass key. Our performance evaluation and computer simulation results indicated that LiBAC is lightweight and suitable to be deployed in healthcare IoT networks.

References

[1] M. Ma, D. He, N. Kumar, K.-K. Raymond Choo, and J. Chen, "Certificateless searchable public key encryption scheme for industrial internet of things," *IEEE Trans. Ind. Informat.*, 2017, DOI: 10.1109/TII.2017.2703922.

[2] P. Hu, H. Ning, T. Qiu, Y. Zhang, and X. Luo, "Fog computing-based face identification and resolution scheme in internet of things," *IEEE Trans. Ind. Informat.*, vol. 13, no. 4, pp. 1910–1920, Aug. 2017.

[3] F. Fernandez and G. C. Pallis, "Opportunities and challenges of the internet of things for healthcare: Systems engineering perspective," in *Proc. EAI 4th Int. Conf. IEEE Wireless Mobile Commun. Healthcare (Mobihealth)*, 2014, pp. 263–266.

[4] G. Yang *et al.*, "A health-IoT platform based on the integration of intelligent packaging, unobtrusive bio-sensor, and intelligent medicine box," *IEEE Trans. Ind. Informat.*, vol. 10, no. 4, pp. 2180–2191, Nov. 2014.

[5] J. Li, L. Huang, Y. Zhou, S. He, and Z. Ming, "Computation partitioning for mobile cloud computing in big data environment," *IEEE Trans. Ind. Informat.*, vol. 13, no. 4, pp. 2009–2018, Aug. 2017.

[6] Q. Zhang, L.T. Yang, and Z. Chen, "Privacy preserving deep computation model on cloud for big data feature learning," *IEEE Trans. Comput.*, vol. 65, no. 5, pp. 1351–1362, May 2016.

[7] D. Kotz *et al.*, "Security for mobile and cloud frontiers in healthcare," *Commun. ACM*, vol. 68, no. 8, pp. 21–23, 2015.

[8] R. Ostrovsky, A. Sahai, and B. Waters, "Attribute-based encryption with non-monotonic access structures," in *Proc. 14th ACM Conf. Comput. Commun. Security*, 2007, pp. 195–203.

[9] J. Lai, R. H. Deng, C. Guan, and J. Weng, "Attribute-based encryption with verifiable outsourced decryption," *IEEE Trans. Inf. Forensics Security*, vol. 8, no. 8, pp. 1343–1354, Aug. 2013.

[10] B. Qin, R. H. Deng, S. Liu, and S. Ma, "Attribute-based encryption with efficient verifiable outsourced decryption," *IEEE Trans. Inf. Forensics Security*, vol. 10, no. 7, pp. 1384–1393, Jul. 2015.

[11] J. Ning, X. Dong, Z. Cao, L. Wei, X. Lin, "White-box traceable ciphertext-policy attribute-based encryption supporting flexible attributes," *IEEE Trans. Inf. Forensics Security*, vol. 10, no. 6, pp. 1274–1288, Jun. 2015.

[12] J. Zhou, Z. Cao, X. Dong, and X. Lin, "TR-MABE: White-box traceable and revocable multi-authority attribute-based encryption and its applications to multi-level privacy-preserving e-healthcare cloud computing systems," in *Proc. IEEE Conf. Computer Commun.*, 2015, pp. 2398–2406.

[13] Y. Rouselakis and B. Waters, "Efficient statically-secure large-universe multi-authority attribute-based encryption," in *Proc. Int. Conf. Financ. Cryptogr. Data Security.*, 2015, pp. 315–332.

[14] Yang, Yang, Xianghan Zheng, and Chunming Tang, "Lightweight distributed secure data management system for health internet of things," *J. Netw. Comput. Appl.*, vol. 89, pp. 26–37, 2017.

[15] H. Deng *et al.*, "Ciphertext-policy hierarchical attribute-based encryption with short ciphertexts," *Inf. Sci.*, vol. 275, pp. 370–384, 2014.

[16] E. Luo, Q. Liu, and G. Wang, "Hierarchical multi-authority and attribute-based encryption friend discovery scheme in mobile social networks," *IEEE Commun. Lett.*, vol. 20, no. 9, pp. 1772–1775, Sep. 2016.

[17] Z. Wan, J. Liu, R. Deng, "HASBE: A hierarchical attribute-based solution for flexible access control in the cloud computing," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 2, pp. 743–754, Apr. 2012.

[18] S. Wang *et al.*, "An efficient file hierarchy attribute-based encryption scheme in cloud computing," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 6, pp. 1265–1277, Jun. 2016.

[19] C. Li *et al.*, "A practical construction for large universe hierarchical attribute-based encryption," *Concurrency Comput.: Pract. Experience*, vol. 29, no. 17, pp. 1–10, 2016.

[20] A. D. Brucker and H. Petritsch, "Extending access control models with break-glass," in *Proc. 14th ACM Symp. Access Control Models Technol.*, 2009, pp. 197–206.

[21] A. D. Brucker, H. Petritsch, and G. W. Stefan, "Attribute-based encryption with break-glass," in *Proc. 4th IFIP WG 11.2 Int. Conf. Inf. Security Theory Pract.: Security Privacy Pervasive Syst. Smart Devices*, 2010, pp. 237–244.

[22] S. Marinovic *et al.*, "Rumpole: A flexible break-glass access control model," in *Proc. 16th ACM Symp. Access Control Models Technol.*, 2011, pp. 73–82.

[23] S.-W. Sigrid and M. Strembeck, "Generic support for RBAC break-glass policies in process-aware information systems," in *Proc. 28th Annu. ACM Symp. Appl. Comput.*, 2013, pp. 1441–1446.

[24] H. A. Maw, H. Xiao, B. Christianson, and J. A. Malcolm, "BTG-AC: Break-the-glass access control model for medical data in wireless sensor networks," *IEEE J. Biomed. Health Inf*, vol. 20, no. 3, pp. 763–774, May 2016.

[25] T. Zhang, S. C. Sherman, and J. Sun, "Password-controlled encryption with accountable break-glass access," in *Proc. 11th ACM Asia Conf. Comput. Commun. Security*, 2016, pp. 235–246.

[26] A. Beimel, "Secure schemes for secret sharing and key distribution," Ph.D. dissertation, Dept. Computer Science, Israel Inst. Technology, Technion, Haifa, 1996.

[27] B. Lynn, *The Stanford Pairing Based Crypto Library*. Nov. 22, 2017. [Online]. Available: http://crypto.stanford.edu/pbc. Accessed on: Jul. 12, 2017.